Source: docs\RAZORPAY_TEST_QA_FINDINGS.md | Export: 11 November 2026

Razorpay TEST QA findings and regression protection

Audit date: 8 November 2026. Project `parinaya-silks`, Firestore `(default)`. Local starting commit: `7c37b00` (`Razorpay fix`). Changes in this report are local only.

Evidence boundaries

The owner confirmed successful and failed TEST card payments, failed-to-successful retry, interrupted-tab recovery, manual FIRST application, and nonzero loyalty redemption on desktop/mobile. These are owner-performed hosted transactions, not transactions initiated by this audit.

Trusted read-only production inspection at 17:49 IST found 10 TEST attempts, all now captured/paid/Confirmed with consumed reservations. Three retain an earlier failure observation. Every order has exactly one inventory movement of -1 for its reservation, with no second deduction at capture. This bounded sample was not truncated; it does not prove every historical or provider-side payment is unique. No currently failed/expired attempt remained for a hosted release observation.

The worker was in TEST mode with a 36-second-old heartbeat, within the five-minute requirement. No authenticated worker, status, reconcile, payment, refund or business-record write was invoked in production. Vercel identified the current alias as READY Production, deployment `dpl_ENxm3M339k1sPvG1HqVuvGJkhSaV`; its inspection response did not expose a commit SHA. Local fixes have not been deployed.

Findings and focused fixes

Failed online payment presentation

`server/orders.ts` intentionally saves an initiated online checkout as lifecycle `Placed`, payment status `pending`, and an active reservation. That is a durable retry record, not a payment confirmation. `server/payments/settlement.ts` confirms/marks paid only after provider capture, amount, currency and order mapping are verified. Admin cannot manually confirm an uncaptured online order. Failed payment observations preserve the retry record.

Customer and Admin lifecycle badges previously displayed the raw `Placed` value. Customer progress also treated it as fulfilment progress, making an unpaid attempt look accepted. Shared payment-aware presentation now shows Awaiting payment / Payment failed / Awaiting capture / Verifying payment / Payment needs review / Payment expired before fulfilment. Customer details and My Orders hide premature shipment/progress content, expose the existing retry action when allowed, and never celebrate an unconfirmed payment even on a confirmation URL. Captured orders and COD retain their existing lifecycle presentation. No stored order schema or payment transition was changed.

Inventory reservation versus consumption

The observed -1 is the existing temporary checkout reservation. Retry reuses its provider order and reservation. Authoritative capture consumes that reservation without another stock change. The reservation lasts 30 minutes from checkout initiation; after expiry the scheduled worker checks provider evidence and releases uncaptured, unambiguous attempts transactionally. Release restores stock, coupon usage and redeemed points once, marks the attempt Cancelled/released, and clears its checkout lock.

Authorized payments, unknown provider-order creation, late capture after release, provider outage and inconsistent provider evidence are held/deferred for safe reconciliation or review. They are not released blindly. Worker execution can add delay beyond the expiry timestamp. Customer copy now explains the reservation deadline and review exception. Production inventory was not adjusted.

A new emulator regression exercises failed attempt -> retry -> trusted expiry -> repeated worker/reconciliation, asserting one reservation, one release, no confirmation/collection, restored coupon usage, and restored nonzero loyalty balance. Existing tests cover overselling, late capture, unknown responses, capture after commit failure, repeated callbacks and delayed failed events after capture.

FIRST missing from Available Coupons

Production FIRST is enabled but explicitly has `discoverable: false` (Show in Available Offers off). It is 30 percent off with a minimum subtotal of 9,900 paise, no dates, usage limits or Delivered-order sequence restriction. Both sampled test accounts were manually eligible; discovery correctly omitted the private code. This is a visibility configuration issue, not a discount-validation failure.

No private code was published automatically. Admin coupon rows now distinguish Listed in Available Offers from Private code - manual entry only. Empty checkout discovery explains that private codes can still be entered. An API/emulator regression reproduces the real configuration: private FIRST applies manually, stays absent from discovery, then appears only after an authorized Admin save enables discovery. Production FIRST remains unchanged.

Owner action if FIRST should be public: Admin -> Coupons -> FIRST -> Edit -> enable **Show in Available Offers** -> Save. Active status alone does not make a coupon public. Existing eligibility coverage includes inactive, expired, future, minimum value, global/customer usage limits, Delivered sequence, token ownership, stale eligibility, and final server-side revalidation. No product/category-specific coupon restrictions are implemented by the current coupon schema; none were invented.

Webhook, duplicate and other-method acceptance

Production read-only records contain processed events: payment.authorized 10, payment.captured 10, payment.failed 3, order.paid 10. All three refund event types have zero records. These corroborate actual owner TEST transactions; original signed bytes are not retained for independent replay/HMAC recalculation. No hosted replay was triggered.

Existing signed synthetic tests exercise all seven supported events, original-byte signature rejection on tampering, duplicate event IDs, conflicting bodies, repeated callbacks/workers, and single confirmation/stock consumption. A refund event alone cannot invent provider refund evidence. Existing refund tests were retained; no refund feature or transaction was added. Hosted refund acceptance is deferred.

UPI and netbanking real TEST acceptance remain outstanding. Official Standard Checkout docs support test UPI IDs (`success@razorpay`, `failure@razorpay`) and a mock-bank success/failure page for netbanking. UPI Intent/QR need LIVE mode; TEST UPI cancellation is not representative (the documented cancellation path succeeds). Do not activate LIVE to exercise these. Use only methods actually offered by this account's TEST Checkout, approved accounts and dedicated inventory in a later owner-run acceptance session.

References: Standard Checkout TEST methods, TEST UPI details.

Validation and release

All 11 required commands passed. Automated gateway traffic uses synthetic providers/intercepts and loopback Auth/Firestore emulators; it is not real Razorpay payment acceptance. No tests were disabled or weakened. The expanded empty-offers copy was separated from its existing status sentence after a focused browser check caught an exact-text mismatch.

No production configuration, users, orders, inventory, coupons, loyalty, rules or indexes were changed. TEST mode/activation settings were left untouched. The literal LIVE activation value was not read from masked deployment secrets; the owner must retain `false`. No commit, push or deployment was performed.

Required remaining acceptance: owner review and authorized deployment of local presentation changes; failed/pending/captured customer and Admin views after release; hosted expiry/release and duplicate replay with approved fixtures; UPI/netbanking if offered. FIRST visibility can be changed separately by its owner. Refund acceptance remains deferred.

Commands completed

Check	Result
`npm run test`	187 tests passed; zero failures or skips
`npm run lint`	Typecheck passed
`npm run build`	Production frontend build passed
`npm run test:catalog-runtime`	Packaged API runtime passed
`npm run test:payments-runtime`	Packaged Web entrypoints passed
`npm run test:foundation`	Full suite passed
`npm run test:analytics`	Consent and page-view suite passed
`npm run test:admin`	Full rules/service suite and all 30 browser files passed
`npm run test:client-changes`	Full 14 service files and 18 browser files passed
`npm run test:discovery`	Full suite passed
`npm run test:loading`	Full suite passed

The payment emulator suite also exercises the compiled Web entrypoint with original-byte HMAC acceptance and tamper rejection. Runtime packaging targets Vercel Node 24; local execution used Node 22.22.2, not a hosted Node 24 invocation. The payment browser suite verifies 320/390/768/1440px and failed-payment list/details/confirmation-URL behavior at 390/1440px, without real gateway requests. Other complete browser suites include six-width desktop/mobile checks.

Files changed

- Payment presentation: `src/payments/model.ts`, `PaymentStatus.tsx`, `src/orders/CustomerOrderHistory.tsx`, `CustomerOrderPage.tsx`, `customerPresentation.tsx`, `historyModel.ts`, `customerOrders.css`, `history.css`.
- Admin labels: `src/admin/orders/OrdersList.tsx`, `OrderDetails.tsx`, `src/admin/coupons/CouponsPage.tsx`.
- Coupon guidance: `src/offers/AvailableCoupons.tsx`.

- Regressions: `src/payments/security.test.ts`, `scripts/payments-emulator.test.ts`, `theme-offers-emulator.test.ts`, `payments-browser.test.py`.
- This report.

No payment backend implementation, environment file, security rule, dependency or generated artifact is part of the intended change.